Update Days
Power Platform

#UDPP26

Developer Autonomy for Power Platform

Terraform-Powered Workspaces

Raphael Pothin

Lead Power Platform Reliability Engineer @Manulife

About Me

Raphael Pothin

Microsoft BizApps MVP

Lead Power Platform Reliability Engineer @ Manulife

Power Platform | IaC & Terraform | Security | Developer Experience | Platform Engineering

rpothin on GitHub

Raphael Pothin on LinkedIn

Building on PPCC25

At PPCC25, I presented how Power Platform governance can be fully code-driven:

- 🛡️ **DLP policies as code** — export existing policies, create new ones, import resources into state
- 🏛️ **Environment groups + Managed Environments** — provisioning with centralized, version-controlled settings
- 🌐 **Azure VNet integration** — enterprise policies, private endpoints, zero-trust networking per environment
- 🔒 **"No touch production"** — the guiding principle we carry forward

> 🚀 ***Environments → our workspace launchpad*** *— same Terraform, same principles but now with the goal to power developer self-service.*

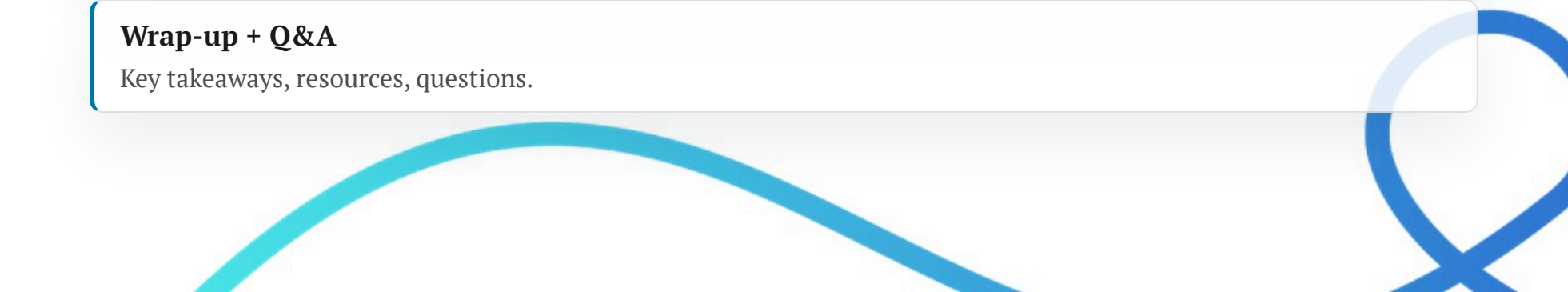
Agenda

Context & Terraform Essentials
Just enough Terraform core concepts to follow along.

Composable Workspace Provisioning
Three patterns that stack. Self-service request → complete workspace. Live demo.

Innersource Extension with AI Agents
Safe contribution with AI coding agents. Multiple tools, one validation pipeline.

Wrap-up + Q&A
Key takeaways, resources, questions.



Why IaC for Power Platform?

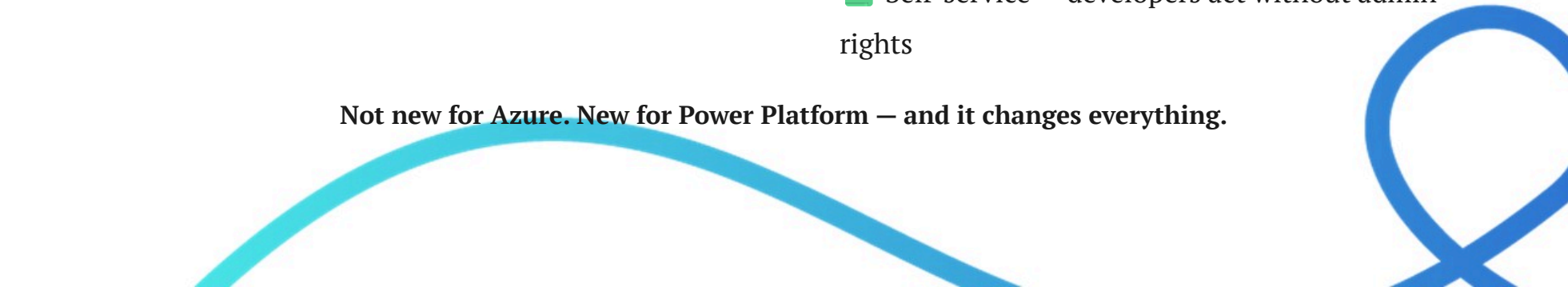
Manual provisioning breaks at scale:

- ❌ Portal clicks — no review, no audit trail
- ❌ Inconsistent — every admin does it differently
- ❌ Drift — you can't prove what is deployed
- ❌ Bottleneck — only admins act, developers wait

IaC changes the model:

- ✅ Declarative — describe desired state, not steps
- ✅ Version-controlled — every change reviewed and traced
- ✅ Repeatable — same input, same output, every time
- ✅ Self-service — developers act without admin rights

Not new for Azure. New for Power Platform — and it changes everything.



Terraform Essentials

Core concepts — just enough to follow along:

- **Provider** — to interact with an API (`microsoft/power-platform`, `azurerm`)
- **Resource** — infrastructure item declared in `.tf`, Terraform keeps it in sync
- **Variables** / `.tfvars` — parameterize a config, one file per workspace = the self-service key
- **State** — Terraform's memory, stored remotely, separate state = separate blast radius
- **Outputs + Remote State** — patterns expose values, downstream patterns consume them across state boundaries

```
variable "project_name" { type = string }

resource "powerplatform_environment" "dev" {
  display_name     = "${var.project_name}-dev"
  location         = "unitedstates"
  environment_type = "Sandbox"
}

output "environment_id" {
  value = powerplatform_environment.dev.id
}
```

📋 `init` → `validate` → `plan` → `apply` — you will see these in every pipeline.

🔐 **OIDC** — no secrets stored, no rotation risk. Non-negotiable.

Composable Workspace Provisioning

Three patterns that stack — reliable, repeatable, no-touch production

Composable Pattern Architecture

🧱 `ptn-workspace` — **Base Layer**

Template-driven environments (basic / simple / enterprise), environment group, settings.

🔗 `ptn-workspace-alm-engine` — **ALM Layer**

Deployment pipeline + stages, deployment environment records, GitHub repository, soft-delete control.

🌐 `ptn-workspace-vnet-extension` — **Network Layer**

Per-environment resource groups, primary + failover VNets, NSGs, DNS zones, enterprise policies.

💡 **Each pattern is independent. Compose what you need. Extend when requirements grow.** Each has its own Terraform state — separate lifecycle, separate blast radius.

Pattern: `ptn-workspace`

What it provisions:

- `module "environments"` — template-driven: basic (Dev/Test/Prod), simple (Dev/Prod), enterprise (Dev/Staging/Test/Prod)
- `module "environment_group"` — governance from day one, no orphaned environments
- `module "environment_settings"` — per-environment audit, security, feature settings
- `module "environment_application_admin"` — monitoring service principal on all environments

The foundation:

```
# One file per workspace — choose your template
workspace_template = "basic"  # Dev + Test + Prod
name               = "CustomerPortal"
location           = "europe"
security_group_id  = "[AAD-GROUP-ID]"
```

Four variables. That is all a developer fills in. Terraform does the rest.

Pattern: `ptn-workspace-alm-engine`

What it provisions:

- `module "deployment_environments"` — workspace environments registered in the Power Platform pipelines host
- `module "deployment_pipeline"` — deployment pipeline + stages with `lifecycle_state` control
- `module "solutions_repo"` — GitHub repository for solution source control
- Remote state consumption from `ptn-workspace` via `paired_tfvars_file`

Paired to workspace by tfvars file name:

```
data "terraform_remote_state" "workspace" {
backend = "azurerm"
config = {
  key="ptn-workspace-${var.paired_tfvars_file}.tfstate"
}
}
```

Anti-corruption layer ensures loose coupling. Workspace changes do not break ALM config.

> `lifecycle_state = "inactive"` — archives the repo and deactivates pipelines without destroying them. Safe decommission path.

Pattern: `ptn-workspace-vnet-extension`

What it provisions:

- Resource groups + primary **and** failover VNets per environment
- NSGs with zero-trust rules (deny internet in/out by default)
- Private DNS zones for private endpoint connectivity
- Enterprise policies for Power Platform VNet injection

Developer interface — same `paired_tfvars_file` as ALM engine:

```
paired_tfvars_file = "customer-portal"

production_subscription_id     = "<prod-sub-id>"
non_production_subscription_id = "<nonprod-sub-id>"

network_configuration = {
  primary  = { location = "canadacentral", ... }
  failover = { location = "canadaeast",    ... }
}

enable_zero_trust_networking = true
enable_vnet_peering          = false
```

Orchestrated Provisioning

1 Developer commits a tfvars file per pattern

Same name across all 3 patterns — `demo-prep.tfvars` in each `tfvars/` folder.

2 Trigger `workspace-provisioning.yml` via `workflow_dispatch`

Input: tfvars file name. Validates all 3 files exist before proceeding.

3 Sequential plan + apply: workspace → VNet → ALM engine

`plan` first, apply only on detected changes. Each pattern independently skippable.

4 Complete workspace in minutes

Environments, network, ALM — all provisioned. Order enforced by workflow job dependencies.

Workspace Lifecycle

Provisioning →

1. 📁 Commit — tfvars files committed to repo
2. 🚀 Trigger — `workflow_dispatch` with tfvars name
3. ✅ Validate — all 3 tfvars files confirmed
4. 🧱 Apply `ptn-workspace`
5. 🌐 Apply `ptn-workspace-vnet-extension`
6. 🔗 Apply `ptn-workspace-alm-engine`
7. 🚀 Ready — workspace available

← Decommissioning

1. 🚀 Trigger — type `DECOMMISSION` to confirm
2. ✅ Validate — confirmation + tfvars files
3. 🔒 Deactivate ALM — repos archived, pipelines inactive
4. 🌐 Destroy VNet — networking cleaned
5. 🧱 Destroy workspace — environments removed
6. 🧹 Clean — full audit trail preserved

Every lifecycle event goes through git. **Full audit trail from creation to decommission.**

🎯 **Demo Time**

The Developer Experience Gap

The reality we just solved:

- 📝 Submit a ticket → wait 3–5 days for environment setup
- 🔧 IT configures manually → inconsistent results every time
- 🔗 ALM setup requires additional portal work after provisioning
- 🔁 Every new project follows the same slow, error-prone cycle

Making it self-service and accessible:

- 📋 **Request via GitHub** — submit a simple issue, no complex tickets
- 🔁 **Automated Provisioning** — Terraform provisions everything through GitHub workflows
- 🔒 **No-Touch Production** — all changes go through code review before being applied
- ♻️ **Repeatable & Auditable** — same request, same result, every time

🎯 **Demo Time**

Innersource Extension with AI Agents

Enabling safe contribution — anyone can extend, everyone stays safe

But Why Even Consider This?

A platform engineering team can build great Terraform patterns. But they cannot scale infinitely to serve every team's evolving needs.

🏗️ One team, many teams

Dozens of product teams — each with unique requirements and timelines.

🔄 Platform never stops evolving

Power Platform ships new capabilities every release. Patterns need to keep up.

⏳ Backlog always wins

Requests outpace capacity. The bottleneck slows everyone down.

🛡️ Guardian

Sets standards. Owns quality. Reviews every contribution. Never compromises on security or compliance.

🔧 Enabler

Provides tooling. Builds guardrails. Creates contribution paths. Multiplies impact across every team.

Innersource: the model that lets one platform team scale to the entire organization.

The Innersource Challenge

> *"An engineer needs environment settings not yet exposed in the workspace pattern. Or wants to expose more variables from the environment resource. How do we enable this — safely?"*

Without guardrails:

- 🚫 Free-for-all contributions
- 🚫 No automated validation
- 🚫 Broken configs reach production
- 🚫 Trust erodes → contribution stops

With innersource guardrails:

- ✅ AI agents follow repo instructions
- ✅ Every change validated automatically
- ✅ Automated testing catches issues early
- ✅ Safe contribution at scale

Safety Guardrails

📜 AGENTS.md

Repository-level instructions for humans and AI agents alike. Coding standards, Terraform conventions, security rules, and stop conditions. The shared constitution.

🤖 `.github/agents/`

Two built-in agents: `terraform-implementer` contributes to `configurations/`, `pr-reviewer` reviews PRs as a technical lead. Each scoped to its domain.

⚙️ Validation Workflows

`terraform-validation-detect-and-dispatch.yml` detects changed paths. `terraform-single-path-validation.yml` validates each independently.

👤 Human Review

AI proposes, human approves. Every PR requires review before merge. The agent is a contributor, not an admin.

Validation Pipeline: Detect & Dispatch

1 Detect

`terraform-validation-detect-and-dispatch.yml` scans the PR for changed configuration paths.

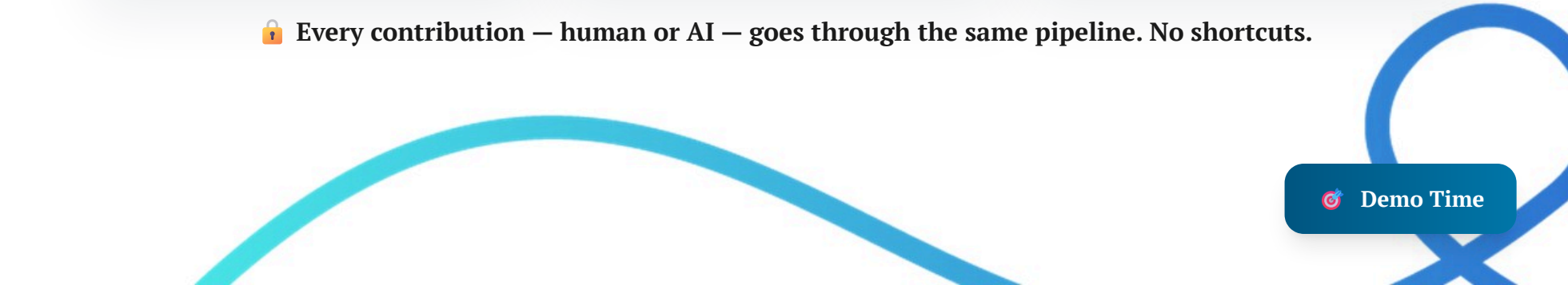
2 Dispatch

For each changed path, triggers `terraform-single-path-validation.yml` independently. Parallel validation.

3 Validate

Five stages per path: AVM compliance check → format check → `terraform init` + `validate` → security scan (Trivy) → `terraform test`. Real tests, not just a plan.

🔒 **Every contribution — human or AI — goes through the same pipeline. No shortcuts.**



🎯 Demo Time

Key Takeaways

Composable workspace provisioning through git

Three patterns that stack: workspace → VNet → ALM engine. Reliable, repeatable, no-touch production.

Lifecycle management as code

Provisioning and decommissioning both go through git. Full lifecycle, full audit trail.

Innersource extension with AI coding agents

AGENTS.md + scoped agent definitions + a 5-stage validation pipeline + human review. Any AI tool, any contributor — same guardrails, every time.

Foundation for an internal developer platform

Fork the open-source reference. Composable patterns + innersource model = a potential starting point for your Power Platform IDP.

Update Days
Power Platform

#UDPP26

Thank You!

RaphaelPothin

Raphael POTHIN

rpothin

Resources & References

Demo repository:

- udpp26-power-platform-devex-with-terraform

Power Platform Terraform Provider:

- registry.terraform.io/providers/microsoft/power-platform

Previous session (PPCC25):

- Session demonstrations recording on YouTube
- PPCC25 demo repository

Background reading:

- IaC for Power Platform — a light at the end of the tunnel?

Connect:

- GitHub: rpothin · LinkedIn: Raphael Pothin

Update Days
Power Platform

#UDPP26

Session: #21

GIVE US FEEDBACK FOR THIS SESSION

Deploying Power Platform and Azure infrastructure as code